DookuG

Table of Contents

1. Általános . . . 1
2. Tartalom . . . 2
3. DookuG modul kliens . . . 2
   3.1. Általános . . . 2
   3.2. Konfiguráció . . . 2
      3.2.1. A következő értékek megadása célszerű a kliens működéséhez: . . . 3
   3.3. Kliens használata . . . 3
      3.3.1. Kliens metódusok . . . 4
      3.3.2. Saxon(-HE) engine használata a kliensben . . . 9
      3.3.3. Hibakezelés . . . 10
4. DOCUMENT service . . . 10
   4.1. Dokumentum generálása . . . 10
      4.1.1. Dokumentum generálása request body alapján . . . 11
      4.1.2. Dokumentum generálása multipart form alapján . . . 12
      4.1.3. Dokumentum generálása mentett sablon alapján . . . 13
   4.2. Dokumentum metaadatainak lekérdezése . . . 15
   4.3. Dokumentum lekérdezése . . . 16
   4.4. Template cache . . . 17

Version: {release_version}

A projekt célja hogy általános template alapú dokumentum generálást tegyen lehetővé a kliensei számára.

A projekt microservice alapon tartalmaz minta service-ket:

- REST Service - Csak REST szükségleteket szolgál ki.

1. Általános

A technológiák melyeken alapszik:

Service (jar, war)

- Java 11+
- Maven 3.3.0+
- Java EE 8
- Wildfly 26.1.1Final

Környezet

- Docker compose, Docker

2. Tartalom

Projekt struktúra

- `/docs` - Dokumentációk tárhelye
- `/etc` - Konfigurációk, fejlesztési környezet scriptek, stb.
- `/dookug-bom` - Maven dependency definíciók
- `/dookug-api` - modul API
- `/dookug-common` - Közös, általános kódok melyeket a service-k használnak
- `/dookug-client` - Kliens a modulhoz
- `/dookug-engine` - Feldolgozást segítő tool implementációk
- `/dookug-document` - Dokumentum generálást összefogó REST szolgáltatások
- `/dookug-testsuite` - Fejlesztői tesztesetek gyűjtő könyvtára

3. DookuG modul kliens

A kliens célja a DookuG modul kezelésének támogatása, amely különböző kiszolgáló metódusokat tartalmaz.

3.1. Általános

Használt technológiák:

- Java 11
- CDI 2.0
- commons-lang3 3.9
- Microprofile restclient 2.0
- Microprofile config 2.0
- Bouncycastle 1.61 (wildfly provided)

3.2. Konfiguráció

Microprofile Config segítségével tudjuk a kliens működését befolyásolni.

```
dookug:
    client:
        document/mp-rest/url: http://localhost:8082
        document/mp-rest/connectTimeout: 5000 #millisec
```

```
        document/mp-rest/readTimeout: 60000 #millisec
```

3.2.1. A következő értékek megadása célszerű a kliens működéséhez:

Paraméter kulcs	Kötelező	Leírás
dookug.client.document/mp-rest/url	igen	a dookug module elérési útvonala (base url)
dookug.client.document/mp-rest/connectTimeout		Connection Timeout megadása (ha coffee-t használunk, ez alapértelmezetten 5 másodperc)
dookug.client.document/mp-rest/readTimeout		Read Timeout megadása (ha coffee-t használunk, ez alapértelmezetten 1 perc)
dookug.service.cache.template.ttl	nem	Template cache lejárati ideje percben, alapértelmezetten 60 perc
dookug.service.cache.template.enablestatistic	nem	generálódjanak metrikák, alapértelmezetten nem generálódnak

További paraméterek megadása is lehetséges, lásd a Microprofile RestClient dokumentációban.

3.3. Kliens használata

A kliens használata mindössze egy pom.xml bejegyzés,

```
<dependency>
   <groupId>hu.icellmobilsoft.dookug</groupId>
   <artifactId>dookug-client</artifactId>
   <version>${version.dookug.client}</version>
</dependency>
```

majd egy @Inject,

```
@Inject
private DookugClient dookugClient;
```

mely az összes végponthoz definiál kliens hívást, és használhatjuk is az általa kínált metódusokat. Microprofile rest klienst használ az API híváshoz így a beállítások a már megszokott microprofile-config segítségével végezhetőek el. (lásd később)

WARNING	A kliens alkalmazás feladata, hogy a konténerbe mountolja be a `/home/icellmobilsoft/resources` mount point alá az általa használt resource-

okat, melyekre a template-ekben hivatkozik!

3.3.1. Kliens metódusok

A kliensnek jelenleg több metódusa is van, melyekkel dokumentumot lehet generáltatni, a létrehozott dokumentumok metaadatait lehet lekérdezni, illetve magát a generált dokumentumot lehet lekérdezni.

A dokumentum generálása során a metódusok egy részénél a generált dokumentumot kapjuk vissza:

```
public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates, Collection<ParameterType> parameters) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates, ParametersDataType parametersData) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateEntityBody(Collection<TemplateType>
templates) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template,
Collection<ParameterType> parameters) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template,
ParametersDataType parametersData) throws BaseException;

public GeneratedDocumentDto postDocumentGenerateMultipart(InputStream template) throws
BaseException;

public GeneratedDocumentDto postDatabaseStoredTemplateDocumentGenerate(String
templateName, OffsetDateTime templateValidity, Collection<ParameterType> parameters)
throws BaseException;

public GeneratedDocumentDto postDatabaseStoredTemplateDocumentGenerate(String
templateName, OffsetDateTime templateValidity, ParametersDataType parametersData)
throws BaseException;
```

Fontos kiegészítés, hogy a `ParametersDataType` típusú paramétereket bár kézzel is létrehozhatjuk, de készült hozzájuk egy ParametersDataBuilder segédosztály, amiben fluent API segítségével állíthatjuk be a kívánt konfigurációkat. Létrehozását a SAXON template motor kívánta meg, mert az XSLT template-ek generálásához szükséges megadni további paramétereket is, melyek nélkül nem generálható dokumentum. A builderből SAXON engine-hez nagyon könnyen kinyerhető a kívánt konfiguráció a getSaxonParameters(byte[], byte[], boolean) metódus meghívásával, melynek a FOP konfiguráció tartalma, az XML dataset és az XML tömörítettségét jelző értéket lehet megadni, vagy az overload-olt metódusokon keresztül csak az XML fájl tartalmát.

```
GeneratedDocumentDto resp = client.postDatabaseStoredTemplateDocumentGenerate(
        TemplateLanguageType.HU, ②
```

```
        ParametersDataBuilder.getSaxonParameters(

FileUtil.readFileFromResource(DocumentServiceTestConstant.XSLT_TEMPLATE_PARAMS).getBytes(StandardCharsets.UTF_8))); ③
```

① XSLT template neve

② A template nyelve

③ Az XML dataset tartalma (jelen esetben tömörítetlenül)

Ezek a metódusok - sikeres hívás esetén - egy GeneratedDocumentDTO-val térnek vissza, mely tartalmazza a generált fájlnevet, a generált objektumot, mint stream-et és a HTTP status kódot.

Paraméterként meg kell kapniuk a template listát, valamint a paramétereket, ha a template-ekben változók vannak, amiket ki kell cserélni értékekre, pl. HANDLEBARS template engine esetén. A paraméterek lehetnek key-value párok, vagy egy JSON objektum, mely tulajdonképp ezeket a key-value párokat tartalmazza.

A metódusok elkülönülnek a dokumentum generáláshoz felhasznált template megadásának formájában. A `postDocumentGenerateEntityBody()` metódusoknál a request body-ban küldjük a sablonként használt struktúrát, a `postDocumentGenerateMultipart()` metódusoknál form paraméterként, InputStream-ként, adható meg a sablon, míg a `postDatabaseStoredTemplateDocumentGenerate()` metódusoknál csak a sablon nevét kell megadni, mivel ezt a megadott tárolási metódusnak megfelelően kezeli a végpont.

Ezeken a metódusokon kívül lehetőség van még olyan dokumentum generálásra, ahol nem magát az elkészült fájlt kapjuk meg, hanem az azt leíró metaadatokat:

```
public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates,
Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates,
ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse
postDocumentGenerateEntityBodyMetadata(Collection<TemplateType> templates) throws
BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template, Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template, ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse postDocumentGenerateMultipartMetadata(InputStream
template) throws BaseException;

public DocumentMetadataResponse
```

```
postDatabaseStoredTemplateDocumentGenerateMetadata(String templateName, OffsetDateTime
templateValidity, Collection<ParameterType> parameters) throws BaseException;

public DocumentMetadataResponse
postDatabaseStoredTemplateDocumentGenerateMetadata(String templateName, OffsetDateTime
templateValidity, ParametersDataType parametersData) throws BaseException;

public DocumentMetadataResponse postStoredTemplateDocumentGenerateMetadata(String
templateName, OffsetDateTime templateValidity, TemplateStorageMethodType
templateStorageMethodType, Collection<ParameterType> parameters, ParametersDataType
parametersData) throws BaseException;
```

Ezek a metódusok - sikeres hívás esetén - egy DocumentMetadataResponse-val térnek vissza, mely tartalmazza a dokumentum egyedi azonosítóját, fájlnevét, tárolási módját, formátumát és a státuszát.

Hasonlóan, az előzőleg felsorolt metódusoknál, a sablon megadásának formájában ugyanúgy különülnek el a kliens hívások.

Ahhoz, hogy láthassuk, hogy már milyen fájlokat generáltunk lehetőség van a dokumentumok metaadatainak lekérdezésére:

```
public DocumentMetadataQueryResponse
postDocumentMetadataQuery(DocumentMetadataQueryParamsType queryParams) throws
BaseException;

public DocumentMetadataQueryResponse
postDocumentMetadataQuery(DocumentMetadataQueryRequest queryRequest) throws
BaseException;
```

A lekérdezés - sikeres hívás esetén - DocumentMetadataQueryResponse-val térnek vissza, mely tartalmazza a megadott paraméterekre illeszkedő dokumentumok metaadatait, valamint a lapozhatósági adatokat: az összes elem számát, az összes oldal számát, az oldal számát, amelyekhez visszaadtuk a dokumentum metaadatait, valamint, hogy hány elemet adtunk vissza a lekérdezésben.

Paraméterként a szűrési feltételeket, illetve a második esetben a lapozási paramétereket is megadhatjuk. Ha ezt nem tesszük, akkor az első oldal 15 elemét kapjuk vissza a válaszban. Szűrési paraméternek megadható a dokumentum neve, formátuma, a dokumentum tárolási metódusa, a sablon azonosítója, amelyből a dokumentumot generáltuk, a dokumentum tárolójának az azonosítója és a dokumentum státusza.

Lehetőség van arra is, hogy a korábban kigenerált dokumentumot kérdezzük le:

```
public GeneratedDocumentDto getDocumentContent(String documentId) throws
BaseException;
```

A válaszban - sikeres hívás esetén - visszaadjuk a generált dokumentumot, mint stream-et, a fájl

nevét, és a HTTP státusz kódot.

Paraméterként a dokumentum egyedi azonosítóját kell megadni.

A kliensnek van pár további beállítási lehetősége is, melyekkel a generálást tudjuk befolyásolni:

```
setGeneratorEngineType()
```

```
setTemplateEngineType()
```

```
setResponseFormat()
```

```
setDocumentStorageMethodType()
```

```
setDigitalSigningType()
```

A `setGeneratorEngineType()` segítségével az output generáláshoz használható engine-t állíthatjuk be, mely jelenleg az alábbiak lehetnek:

- `PDF_BOX` — Apache PdfBox használata
- `SAXON` — Saxon HE használata
- `NONE`

A `setTemplateEngineType()` segítségével template 'típusát' állíthatjuk be, mely jelenleg az alábbiak lehetnek:

- `HANDLEBARS` — Handlebars használata
- `NONE`

A `setResponseFormat()` segítségével a válaszformátumot állíthatjuk be.

- `PDF`
- `STRING`

A `setDocumentStorageMethodType()` segítségével a dokumentum tárolási módját állíthatjuk be.

- `NONE`
- `DATABASE`

A `setDigitalSigningType(digitalSigningType)` segítségével tudjuk vezérelni, hogy a generált PDF dokumentumra kerüljön-e digitális aláírás. A digitalSigning további három paramétert vár, melyek az alábbiak:

- signatureName - az aláírás 'neve' (opcionális)
- signatureReason - milyen célból írta alá (opcionális)
- keyAlias - a keystore-ban lévő kulcs azonosítója, amivel az aláíró kulcsot tudjuk azonosítani (opcionális, de célszerű megadni, mert alapértelmezetten a `test` nevű kulcsot keresi)

Ha nem állítunk be semmit, alapértelmezetten a `PDF_BOX` + `HANDLEBARS` + `PDF` + `NONE` értékek vannak beállítva, digitális aláírás nélkül.

A `postDocumentGenerateEntityBody()` metódusok az alábbi REST végpontot hívják meg a modulban:

```
POST /internal/dookug/document/generate/inline
```

A kliens a kérésben megküldi:

- ContextType-ot
- a kapott TemplateType listát
- a kapott ParameterType listát
- a GeneratorSetup objektumot, amit a kliens `set` metódusaival tudunk kontrollálni.

Ha a kérés megfelelő, akkor visszakapjuk a generált objektumot.

NOTE	A Multipart és StoredTemplate kliensek esetében ugyanígy járunk el, azok csak a REST API hívásokban különböznek.

A `postDocumentMetadataQuery()` metódusok az alábbi REST végpontot hívják meg a modulban:

```
POST /internal/dookug/document/storedTemplate/metadata/query
```

A kliens a kérésben megküldi:

- ContextType-ot
- a kapott szűrési feltételeket
- a lapozási paramétereket
- a sorrendezési beállításokat

Ha a kérés megfelelő, akkor visszakapjuk a kérésben beküldött paraméterekre illeszkedő dokumentum metaadatokat.

Példa a kliens használatához:

```
@Inject
private DookugClient dookugClient;
...
//template objektum
```

```
 TemplateType template = new
TemplateType().withTemplateName("main").withTemplateContent("DookuG client simple test
with prameters first: [{{first}}], second:
[{{second}}]".getBytes(StandardCharsets.UTF_8));

 //paraméterek
 ParameterType parameter1 = new ParameterType().withKey("first").withValue("első");
 ParameterType parameter2 = new
ParameterType().withKey("second").withValue("í189öüóőúűáé-.,<>#&@{};*¤ß$");
 ...
 client.setResponseFormatType(ResponseFormatType.STRING);
 client.setGeneratorEngineType(GeneratorEngineType.NONE);
 GeneratedDocumentDto response =
dookugClient.postDocumentGenerateEntityBody(List.of(template),
List.of(parameter1,parameter2));
```

vagy hasonlóan dokumentum generálása, de PDF dokumentum formátummal, multipart inputtal, és a metaadatot adjuk vissza

```
 @Inject
 private DookugClient dookugMultipartClient;
 ...
 //template byte tömbként
 byte[] template = "DookuG client simple test with prameters first: [{{first}}],
second: [{{second}}]".getBytes(StandardCharsets.UTF_8);


 //paraméterek
 ParameterType parameter1 = new ParameterType().withKey("first").withValue("első");
 ParameterType parameter2 = new
ParameterType().withKey("second").withValue("í189öüóőúűáé-.,<>#&@{};*¤ß$");
 ...
 client.setResponseFormatType(ResponseFormatType.PDF); //ez a default
 client.setGeneratorEngineType(GeneratorEngineType.PDF_BOX); //ez a default
 client.setTemplateEngineType(GeneratorEngineType.HANDLEBARS); //ez a default
 DocumentMetadataResponse response =
dookugMultipartClient.postDocumentGenerateMultipartMetadata(new
ByteArrayInputStream(template), List.of(parameter1, parameter2));
```

3.3.2. Saxon(-HE) engine használata a kliensben

Saxon motor használatához szükséges egy XSLT template, ami alapján egy XML-ből PDF fájlt generálhatunk. (Ebben az esetben csak PDF lehet a kimenet) Szükséges egy fop-config.xml fájlt is átadni a requestben, amivel például a fontok használatát tudjuk szabályozni.

```
<?xml version="1.0" encoding="UTF-8"?>
<fop version="1.0">
    <renderers>
```

```
        <renderer mime="application/pdf">
            <fonts>
                <!-- TTF fonts -->
                <font kerning="yes" embed-
url="/home/icellmobilsoft/fonts/Roboto/Roboto-Regular.ttf">①
                    <font-triplet name="Roboto" style="normal" weight="normal" />
                </font>
                <font kerning="yes" embed-
url="/home/icellmobilsoft/fonts/Roboto/Roboto-Bold.ttf">
                    <font-triplet name="Roboto" style="normal" weight="bold" />
                </font>
            </fonts>
        </renderer>
    </renderers>
</fop>
```

① Itt tudjuk megadni, hogy a fájlrendszerben hol találhatóak meg a használt betűtípusok.

HANDLEBARS szintén használható a SAXON használatakor, ilyenkor a szokásos {{VARIABLE}} változókba helyettesíthetjük be a kívánt szövegrészeket, valamint készíthetünk egymásba ágyazott template-eket is (itt legfőképp erre lehet használni). Ami változás a többi enginehez képest, hogy itt meg kell adni a generatorSetup osztályban az xmlDataToTransform mezőben a transzformálni kívánt XML fájlt, azaz ebben az esetben az XML fájl nem a template lesz, hanem az adatforrás.

Tehát a generatorSetupban SAXON esetén a többi mező mellett kötelezően megadandó:

- XML: mint adatforrás
- XSLT: mint template
- fopConfig: transzformátor konfiguráció

3.3.3. Hibakezelés

A kliens csak BaseExceptiont adhat vissza, de ha az API hívásban egy RestClientResponseException-t kap vissza, akkor az abban lévő becsomagolt BaseException-t fogja visszaadni!

4. DOCUMENT service

- Main: `dookug-document/dookug-document-service`

4.1. Dokumentum generálása

A modul több lehetőséget is biztosít dokumentum generálására. Mindegyik lehetőségnél küldenünk kell, hogy a sablon-t, illetve a dokumentumot milyen engine dolgozza fel, milyen formátumban szeretnénk megkapni a fájlt, illetve milyen tárolási metódussal mentsük el a legenerált dokumentumot. Ha a generált dokumentumunk formátumának nem STRING értéket adunk meg, de nem adunk meg engine-t, amely feldolgozhatja a dokumentumot, illetve ha STRING formátumot szeretnénk, de engine-nek megadtuk a PDF_BOX-t, akkor INVALID_INPUT hibát adunk vissza. A dokumentum generárálásához, ha a kiinduló sablon alapján szükséges, paramétereket is meg kell

adni. Ennek megadására több lehetőség is van, az egyik, hogy kulcs-érték párokban küldjük. A másik esetben egy json szerkezetett várunk el, amit base64binary formában kell küldeni a kérésben.

4.1.1. Dokumentum generálása request body alapján

A generálás során egy request body-ban beküldjük a kiinduló sablont, illetve a generáláshoz tartozó adatokat: sablont és a dokumentumot feldolgozó engine-t, a válasz formátumot, illetve a dokumentum tárolási metódusát.

Minta dokumentum generálás request - json paraméterekkel base64binary formában (a jobb olvashatóság érdekében a sablonok tartalmának és a paraméter értékei le lettek vágva):

```
Request method: POST
Request URI:    http://localhost:8082/internal/dookug/document/generate/inline
Headers:        Accept=application/json
                Content-Type=application/json
```


```
{
  "context": {
    "requestId": "431AZYLPPS6LJE01",
    "timestamp": "2023-02-22T09:12:23.533Z"
  },
  "generatorSetup": {
    "generatorEngine": "PDF_BOX",
    "templateEngine": "HANDLEBARS",
    "parametersData": "ewogICAgIklOU1RJVFVURV9OQU1FIjogIlNpw7Nmb2tpIGvDs3Jow...",
    "responseFormat": "PDF",
    "documentStorageMethod": "DATABASE"
  },
  "templates": [
    {
      "templateName": "main_template",
      "templateContent": "PCFET0NUWVBFIGh0bWw+CjxodG1sPgo8aGVhZD4KCjxzdHls...",
      "initial": true
    },
    {
      "templateName": "head_template",
      "templateContent": "PHRhYmxlIHN0eWxlPSJoZWlnaHQ6IDE2MXB4OyB3aWR0aDog...",
      "initial": false
    }
  ]
}
```


A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra, hogy ne a dokumentumot kapjuk meg a válaszban, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/inline/metadata
Headers:        Accept=application/json
                Content-Type=application/json
```

4.1.2. Dokumentum generálása multipart form alapján

A generáláshoz be kell küldjük a kiinduló sablont, illetve a generáláshoz tartozó adatokat. Ez utóbbi megegyezik a request body alapján történő dokumentum generálásnál beküldött adatokkal.

Minta dokumentum generálás multipart form request kulcs-érték paraméterekkel:

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/generate/inline/multipart
Headers:        Accept=application/octet-stream
                Content-Type=multipart/form-data

Content-Disposition: form-data; name="REQUEST"
Content-Type: application/json
{
    "context": {
      "requestId": "42ZZBR3LKJZ9IT0X",
      "timestamp": "2023-02-21T12:57:52.113Z"
    },
    "generatorSetup": {
      "generatorEngine": "PDF_BOX",
      "templateEngine": "HANDLEBARS",

"parameters":[{"key":"first","value":"első"},{"key":"second","value":"í123456789öüóőúű
áé-.,<>#&@{};*¤ß$"}],
      "responseFormat": "PDF",
      "documentStorageMethod": "DATABASE"
    }
}
--2b17c723-ec5b-4c99-8dcf-c6d8972c4564
Content-Disposition: form-data; name="TEMPLATE"
Content-Type: application/octet-stream

DookuG simple test with prameters first: [{{first}}], second: [{{second}}]
```

A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra is, hogy ne a dokumentumot kapjuk meg, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

```
Request method: POST
Request URI:
```

```
http://localhost:8082/internal/dookug/document/generate/inline/multipart/metadata
Headers:        Accept=application/json
                Content-Type=multipart/form-data
```

4.1.3. Dokumentum generálása mentett sablon alapján

A generáláshoz be kell küldenünk a sablon nevét, amiből generáltatjuk a dokumentumot, valamint a generáláshoz tartozó adatokat. Ez utóbbi megegyezik a request body alapján történő dokumentum generálásnál beküldött adatokkal, kiegészülve azzal, hogy a sablont milyen tárban tároljuk.

Példa dokumentum generálása mentett sablon alapján request - json paraméterekkel base64binary formában (a jobb olvashatóság érdekében a paraméter értéke le lett vágva):

```
Request method: POST
Request URI:    http://localhost:8082/internal/dookug/document/generate/storedTemplate
Headers:        Accept=application/json
                Content-Type=application/json
```

```
{
  "context": {
    "requestId": "431BIFEOUJ0ODU01",
    "timestamp": "2023-02-22T09:26:45.121Z"
  },
  "generatorSetup": {
    "templateStorageMethod": "DATABASE",
    "template": {
      "templateName": "DEV_TEMPLATE_HANDLEBARS",
      "templateLanguage": "HU",
      "validityDate": "2023-02-22T09:26:45.130074Z"
    },
    "generatorEngine": "PDF_BOX",
    "templateEngine": "HANDLEBARS",
    "parametersData": "ewogICJ0aXRsZSI6ICJwZWxkYSBjaW0iLAogICJjdXJyZW50WWV...",
    "responseFormat": "PDF",
    "documentStorageMethod": "DATABASE"
  }
}
```

A válaszban a generált dokumentumot kapjuk vissza, valamint HTTP header-ben a fájl nevét. Lehetőség van arra, hogy ne a dokumentumot kapjuk meg a válaszban, hanem az azt leíró metaadatokat. Ebben az esetben a request URI, és a HTTP header tér el a kérés beküldésekor.

Az adatbázisban tárolt template kulcsa a templateName és templateLanguage értékéből áll össze.

```
Request method: POST
Request URI:
```

```
http://localhost:8082/internal/dookug/document/generate/storedTemplate/metadata
Headers:        Accept=application/json
                Content-Type=application/json
```

A dokumentum mentése a `documentStorageMethod` paraméter megadásától függ. Két értéket vehet fel jelenleg: `NONE` és `DATABASE` Amennyiben a `NONE` értéket adjuk meg, akkor a dokumentumot nem mentjük el, így a későbbiekben nem is lehet lekérdezni. A `DATABASE` esetében a generált dokumentumot adatbázis táblába mentjük el, és a későbbi lekérdezéskor, onnan olvassuk ki. Az adatbázisba elmentjük még a dokumentumhoz tartozó egyéb adatokat is:

- a kiinduló sablon azonosítóját - abban az esetben, ha nincs elmentve a sablon ezt a paramétert nem töltjük
- a generált fájl nevét - Három részből generáljuk a fájlnevet: a dokumentum egyedi azonosítója, a kiinduló sablon neve, és a generálás időpontja long értéke
- a fájl formátuma
- a dokumentum státusza - DONE, FAILED, PENDING, SYNCING
- a dokumentumhoz tartozó paraméterek
- a dokumentum tárolási formája - `DATABASE` esetén ezt a mezőt is DATABASE értékkel töltjük

---

A generálás során bármilyen formában adjuk meg a kiinduló sablont a válaszban vagy a generált fájlt kapjuk vissza, vagy a dokumentumot leíró metaadatokat, DocumentMetadataResponse típusú objektumként.

Példa DocumentMetadataResponse:

```
{
    "context": {
      "requestId": "42ZZBQ5K7W43FI6W",
      "timestamp": "2023-02-21T12:57:50.888Z"
    },
    "funcCode": "OK",
    "metadata": {
      "documentId": "42ZZBQ3ISCXWVO6V",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "status" : "DONE"
    }
}
```

4.2. Dokumentum metaadatainak lekérdezése

A dokumentum metaadatok lekérdezésének célja, hogy a megadott szűrési feltételeknek megfelelő dokumentum adatokat visszaadjuk.

A végpont biztosítja a lapozhatóságot, vagyis az adatokat több oldalon keresztül kaphatjuk meg. Ehhez a kérésben beküldhetjük, hogy melyik oldal adatait szeretnénk visszakapni, illetve az oldalról mennyi elemet. Ennek megfelelően a válaszban visszaadjuk, hogy összesen hány elem található, illetve, hogy az elemek mennyi oldalon férnek el. Amennyiben nem küldjük ezt a paramétert a végpont alapértelmezetten az első 15 elemet fogja visszaadni.

A végponton a következő szűrési feltételek adhatóak meg:

- templateId - a dokumentum generáláshoz használt sablon azonosítója
- status - a dokumentum státusza
- format - a dokumentum fájl formátuma
- storageMethod - a dokumentum tárolási formája
- storageId - a dokumentum tárolójának egyedi azonosítója
- filename - a dokumentum fájl neve

A rendezési paraméter lehet:

- filename
- documentStorageMethod
- format
- status

A rendezési paraméterekhez meg lehet adni külön-külön, hogy csökkenő, illetve növekvő rendezést szeretnénk. Az előbb említett rendezéseken kívül, van egy alapértelmezett rendezés a dokumentum azonosítójára.

Minta DocumentMetadataQueryRequest:

```
Request method: POST
Request URI:
http://localhost:8082/internal/dookug/document/storedTemplate/metadata/query
Headers:        Accept=application/json
                Content-Type=application/json; charset=UTF-8
```

```
{
    "context": {
      "requestId": "43183LDKQNC2R702",
      "timestamp": "2023-02-22T09:15:14.168Z"
    },
    "paginationParams": {
```

```
      "rows": 10,
      "page": 1
    },
    "queryParams": {
      "status": "DONE",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "templateId" : "MAIN_TEMPLATE"
    }
}
```

Ha a kérésben beküldött paraméterek alapján található dokumentum metaadat, akkor a válaszban egy maximum 100 elemű listában visszakapjuk az adatokat.

Minta DocumentMetadataQueryResponse:

```
{
    "context": {
      "requestId": "43183LDKQNC2R702",
      "timestamp": "2023-02-22T09:15:14.168Z"
    },
    "funcCode": "OK",
    "rowList": {
      "documentId": "43183KXXW2KCI206",
      "storageMethod": "DATABASE",
      "filename": "filename.pdf",
      "format": "PDF",
      "status" : "DONE"
    }
  }
```

4.3. Dokumentum lekérdezése

A végpont célja, hogy a már legenerált és elmentett dokumentumot a megadott azonosító alapján visszaadja.

Minta dokumentum lekérdezés request:

```
Request method: GET
Request URI:
http://localhost:8082/internal/dookug/document/content/43183KXXW2KCI206 ①
Headers:        Content-Type=application/octet-stream
```

① A generált dokumentum azonosítója

Amennyiben a beküldött azonosítóhoz nem található dokumentum, akkor ENTITY_NOT_FOUND hibát adunk vissza.

A válaszban - létező dokumentum azonosító esetén - visszaküldjük a generált dokumentumot, és HTTP header-ben a fájl nevét is.

4.4. Template cache

Az adatbázisban tárolt TEMPLATE adatokat cache-eljük (GUAVA). A cache-k meghatározott ideig élnek, új lekéréskor az idő újra indul.

Konfigurációs paraméterek:

DOOKUG_SERVICE_CACHE_TEMPLATE_TTL paraméterrel adhatjuk meg percben meddig tartsuk a cachben a templateket, alapértelmezetten 60 perc.

DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLESTATISTIC paraméterrel adhatjuk meg, hogy szükséges-e a metrikák generálása , alapértelmezetten false, nem képződnek metrikák.

Az alábbihoz hasonló metrika generálódik:

```
# TYPE application_cache_hit_count gauge
application_cache_hit_count{name="template"} 0.0
# TYPE application_cache_miss_count gauge
application_cache_miss_count{name="template"} 1.0
# TYPE application_cache_size gauge
application_cache_size{name="template"} 1.0

:leveloffset!:

:leveloffset: +1

== Saját betűtípusok használata

A leírás az https://pdfbox.apache.org/[Apache PDFBox] engine-hez készült és a saját
betűtípusok használatáról szól.
[NOTE]
====
Az engine limitációja, hogy csak TTF típusú betűtípusokat tud beágyazni, valamint ha
használni szeretnénk a weight,styles,variants beállításokat, akkor érdemes tudni, hogy
ezeket a pdfbox nem tudja mindig megfelelően emulálni, hacsak nem töltjük be a font
különböző variációit is.

====
A docker-compose fájlban a `fonts` könyvtárat hozzáadjuk `/home/icellmobilsoft/fonts`
-ként az image-ünkhöz, így az ebben a könyvtárban lévő fontokat lokálisan el tudjuk
érni a template-ekből.

Két lehetőségünk van arra, hogy saját betűtípusokat tudjunk használni.

. közvetlenül a CSS-ben töltjük be a fontot
. programozottan töltjük be, majd ezután a CSS-ben meghivatkozzuk a referenciát
(`font-family`) használatával
```

```
Az első opcióban a CSS-ben a `@font-face` at-rule-on belül az `src: url()` leíróban
adhatjuk meg a betűtípus elérhetőségét.

Ebben az esetben internet elérésre lesz szükségünk a generáláskor, ami nem feltétlenül
lehet adott minden esetben, mert tilthatja a vállalat policy-je, hogy a modul külső
url-eket érhessen el.

Ez kivédhető vagy úgy, hogy belső URL-en keresztül lehet elérni a használni kívánt
fontokat, vagy a modulban előre feltöltünk néhány gyakran használt betűtípust
és a lokális elérhetőségét adjuk meg file:// url-lel meghivatkozva.
Ha a modul tartalmazni fog előre betöltött betűtípusokat, akkor célszerű ezeket a
dokumentációban jelezni, vagy egy
végponton visszaadni az információkat.

TEMPLATE:
[source,xml]
----
<style>
    @font-face {
        font-family: 'Cairo';
        font-style: normal;
        src: url(file:/home/icellmobilsoft/fonts/Cairo/Cairo-Regular.ttf); ①
    }

    @font-face {
        font-family: 'IndieFlower';
        font-style: normal;
        src: url(file:fonts/IndieFlower/IndieFlower-Regular.ttf); ②
    }
</style>
----
<1> a `/home/icellmobilsoft/fonts` könyvtárban található Cairo-Regular.ttf betöltése.
<2> Itt szintén a fenti könyvtárból töltjük be a fontot. A `/home/icellmobilsoft` -ot
tekinti a server root-ként, ezért működik a relativ útvonal megadása.

[TIP]
====
Google fontok használata

[source,xml]
----
<style>
    @import url('https://fonts.googleapis.com/css?family=Quicksand&amp;display=swap');
①

    .quicksand { ②
      font-family: 'Quicksand', Arial;
      font-weight: 700;
      font-style: normal;
      font-size: 38px;
```

```
      line-height: 1.15;
      letter-spacing: -.02em;
      color: rgba(0, 0, 0, 0.8);
      -webkit-font-smoothing: antialiased;
    }
</style>
...
<p class="quicksand">Google Quicksand Font úőüöóéáyí</p> ③

----
<1> Quicksand betűtípus importja a https://fonts.google.com/[Google Fonts] -ról
<2> CSS class beállítása a font használatához
<3> Szöveg kiírása a beállított betűtípussal

====



Ha fájlrendszerből programozottan szeretnénk betölteni a betűtípusokat, akkor a
`builder.useFonts` metódusát tudjuk használni a renderkor, melyekre a CSS-ben
hivatkozni tudunk.

JAVA:
[source,java]
----
builder.useFont(new File("fonts/NotoSansThai/NotoSansThai-Regular.ttf"),
"notosansthai-regular"); ①
----
<1> NotoSansThai-Regular.ttf betöltése

TEMPLATE:
[source,xml]
----
<style>
    @font-face {
        font-family: 'notosansthai-regular'; ①
        font-style: normal;
        -fs-font-subset: complete-font; ②
    }
</style>
----
<1> fentebb betöltött betűtípus használata
<2> ez csak példaként van itt, a beállítást nem igazán szükséges használni, mert így a
teljes font beágyazásra kerül, viszont alapértelmezetten csak a subset, ami a helyes
működés.


Ennek van egy olyan hátránya, hogy mindegyik fontot betölti a renderkor, azokat is,
amiket nem használunk a dokumentum template-ben.
```

```
[NOTE]
====
A PDFBox Fonts Wiki https://github.com/danfickle/openhtmltopdf/wiki/Fonts[itt
elérhető]

====



:leveloffset: 1

:leveloffset: +1

== PDF-ek digitális aláírása

A leírás az https://pdfbox.apache.org/[Apache PDFBox] engine-hez készült és a
digitális aláíráshoz egy self signed certificate-et használ, melyet a docker-compose
fájl segítségével adunk hozzá az image-hez `/home/icellmobilsoft/keys/keystore.p12`
fájlként.

A keystore az `/etc/docker-compose/keys` mappában található és 2023. november 7-én az
alábbi paranccsal lett létrehozva:

[source,text]
----
keytool -genkeypair -storepass 123456 -storetype pkcs12 -alias test -validity 10958 -v
-keyalg RSA -keystore keystore.p12
----

[WARNING]
====
Ez egyben azt is jelenti, hogy 2053. november 7-én le fog járni.

====


=== Konfiguráció

Az aláírást az API hívásokban a request GeneratorSetup -jában tudjuk kérni. Itt meg
tudjuk adni
a `signatureName`-et, `signatureReason`-t, valamint az private key alias-t
(`keyAlias`) arra az esetre, ha a keystore
fájl több private kulcsot is tartalmazna.

[source,xml]
----
...
<ns2:generatorSetup>
    <ns2:generatorEngine>PDF_BOX</ns2:generatorEngine>
    <ns2:templateEngine>NONE</ns2:templateEngine>
    <ns2:responseFormat>PDF</ns2:responseFormat>
```

```
    <ns2:addDigitalSignature>
        <ns2:signatureName>TestSuite name</ns2:signatureName>
        <ns2:signatureReason>TestSuite teszt reason</ns2:signatureReason>
        <ns2:signatureProfile>sampleProfile</ns2:signatureProfile>
    </ns2:addDigitalSignature>
        ...
----

Az aláírást a következő konfigurációs kulcsokkal tudjuk testreszabni:
[source,yaml]
----
dookug:
  service:
    engine:
      pdf:
        digitalsign:
          signature:
            sampleProfile: ①
              name: Példa kft. ②
              reason: Hitelesített ②
              keystore: /home/icellmobilsoft/keys/keystore.p12 ③
              keystorePass: 123456 ④
              keystoreType: PKCS12 ④
              keyAlias: kulcs_teszt ⑤
----
<1> a signatureProfile neve
<2> a `name` és a `reason` default értékek arra vannak, ha a request nem tartalmazná
ezen értékeket.
<3> a `keystore` kötelezően megadandó, az aláíró kulcsok gyűjtőhelyét tartalmazza.
<4> a `keystorePass`, `keystoreType` a keystore fájlhoz tartozó jelszót, valamint a
keystore típusát adja meg, ezek szintén kötelezően megadandó értékek.
<5> a keystore-on belül a provát kulcs azonosítója

:leveloffset: 1

:leveloffset: +1

= Telepítés, üzembe helyezés

A Dookug modul a szolgáltatást felhasználni kívánó projekt(ek) környezeteiben elérhető
kell legyen.
Ehhez a service egy-egy példánya - infrastrukturális szükségleteivel együtt - minden
egyes (fejlesztői/teszt/éles) környezetre telepítendő és konfigurálandó.

Ez magában foglalja az alábbit:

* egy relációs adatbázist (Oracle vagy Postgres)

== DookuG Service Konfiguráció

.Oracle
```

```
[source,properties]
----
ORACLE_DS_CONNECTION_URL=jdbc:oracle:thin:@module-dookug-oracle:1521/xepdb1
ORACLE_DS_USERNAME=dookug
ORACLE_DS_PASSWORD=developer
ORACLE_DS_MIN-POOL-SIZE=1
ORACLE_DS_MAX-POOL-SIZE=20
----

.Postgres
[source,properties]
----
POSTGRESQL_DS_CONNECTION_URL=jdbc:postgresql://module-dookug-
postgredb:5432/dookug_db?currentSchema=dookug
POSTGRESQL_DS_USERNAME=postgres
POSTGRESQL_DS_PASSWORD=postgres
POSTGRESQL_DS_MIN-POOL-SIZE=1
POSTGRESQL_DS_MAX-POOL-SIZE=20
----

.Interface
[source,properties]
----
DOOKUG_SERVICE_INTERFACE_PARAMETERSDATA_GZIPPED=true
----

.PDF aláírás
[source,properties]
----
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_NAME="Eredeti dokumentum"
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_REASON="Igazolva I-Cell
Mobilsoft Zrt. által"
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTORE=/home/icellmobilsoft/
keys/keystore.p12
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTOREPASS=123456
DOOKUG_SERVICE_ENGINE_PDF_DIGITALSIGN_SIGNATURE_DEFAULT_KEYSTORETYPE=PKCS12
----

.Saxon
[source,properties]
----
DOOKUG_SERVICE_ENGINE_SAXON_FOPCONFIG=/home/icellmobilsoft/fop-config/fop-config.xml
DOOKUG_SERVICE_ENGINE_SAXON_XSLT_LANGUAGE_VARIABLE=lang
DOOKUG_SERVICE_ENGINE_SAXON_XSLT_LANGUAGE_DEFAULT=HU
----

.Handlebars
[source,properties]
----
DOOKUG_SERVICE_ENGINE_HANDLEBARS_HELPER_JAVASCRIPT_DIRECTORY=/home/icellmobilsoft/hand
lebars/helper/js
```

```
DOOKUG_SERVICE_ENGINE_HANDLEBARS_ESCAPINGSTRATEGY=HTML_ENTITY
----

.Memory cache
[source,properties]

DOOKUG_SERVICE_CACHE_TEMPLATE_TTL Alapértelmezetten 60 perc
DOOKUG_SERVICE_CACHE_TEMPLATE_ENABLESTATISTIC Alapértelmezetten false

=== Kubernetes deployment
* Ajánlott konfiguráció

További kulcsok itt találhatók: https://common.pages-
dev.icellmobilsoft.hu/baseimages/docker-
wildfly/master/#_wildflyoracle_jdbc[wildfly/oracle-jdbc konfiguráció]

[options="header",cols="1,1,1"]
|===
|Parameter |Value |Description
|JAVA_OPTS |-Xms2000m -Xmx2000m -XX:MetaspaceSize=96M -XX:MaxMetaspaceSize=256m
-XX:+PrintCommandLineFlags -XX:+UseG1GC |2G memóriában maximalizáljuk a modul heap
szükségletét, UseG1GC algoritmus használatával.
|ORACLE_DS_MAX_POOL_SIZE/POSTGRESQL_DS_MAX_POOL_SIZE|30 |Datasource pool maximális
méret
//|OPENTRACING_JAEGER_AGENT_HOST|jaeger |Jaeger host
//|OPENTRACING_JAEGER_AGENT_PORT|6831 |Jaeger agent port
//|OPENTRACING_JAEGER_AGENT_SAMPLER_PORT|5778 |Jaeger agent sampler port
//|OPENTRACING_JAEGER_PROPAGATION|JAEGER/B3 |Jaeger propagation
//|JAEGER_SERVICE_NAME|dookug-service |Service name ami a Jaeger felületén is látható
(alapértelmezetten ROOT.war)
//|LOGSTASH_ENABLED|1 |Logstash connector bekapcsolása (default 1)
//|LOGSTASH_HOST|tcp:dookug-logstash |Logstash szerver host (a "tcp:" előtag nem
hagyható el)
//|LOGSTASH_PORT|8001 |Logstash GELF input port
//|LOGSTASH_K8S_NAMESPACE|dookug-service |Logstash logging additionalFields
K8S_NAMESPACE beállítása
//|LOGGING_K8S_NAMESPACE|dookug-service |Log sorban szereplő k8s_namespace kulcs
értéke (log pattern)
|===

.Requests/limits beállítások
[source,yaml]
----
spec:
  containers:
    resources:
      limits:
        cpu: "3"
        memory: 3G
      requests:
        cpu: "3"
```

```
        memory: 3G
----

=== Health - startup/liveness/readiness

A service támogatja a k8s probe-ok használatát.

** Beépített indítási ellenőrzések: database


.http://localhost:9990/health/started
[source,json]
----
{
  "status": "UP",
  "checks": [
    {
      "name": "started-deployment.ROOT.war",
      "status": "UP"
    }
  ]
}
----

.http://localhost:9990/health/live
[source,json]
----
{
  "status": "UP",
  "checks": [
    {
      "name": "empty-liveness-checks",
      "status": "UP"
    }
  ]
}
----

.http://localhost:9990/health/ready
[source,json]
----
{
  "status": "UP",
  "checks": [
    {
      "name": "deployments-status",
      "status": "UP",
      "data": {
        "ROOT.war": "OK"
      }
    },
```

```
    {
      "name": "server-state",
      "status": "UP",
      "data": {
        "value": "running"
      }
    },
    {
      "name": "boot-errors",
      "status": "UP"
    },
    {
      "name": "ready-deployment.ROOT.war",
      "status": "UP"
    }
  ]
}
----




:leveloffset: 1

:leveloffset: +1





:leveloffset: 1

:leveloffset: +1

= Release notes

:leveloffset: +1

== v0.4.0

Projekt kikerült Open source Github-ra.

* maven pom change:
** `<groupId>hu.icellmobilsoft.dookug.common</groupId>` ->
`<groupId>hu.icellmobilsoft.dookug</groupId>`
** `<groupId>hu.icellmobilsoft.dookug.document</groupId>` ->
`<groupId>hu.icellmobilsoft.dookug</groupId>`
** `<groupId>hu.icellmobilsoft.dookug.client</groupId>` ->
`<groupId>hu.icellmobilsoft.dookug</groupId>`

:leveloffset: 1
```

```
:leveloffset: 1
```